



Université du Québec
à Chicoutimi

Jalon 2

8INF970 - Atelier pratique en cybersécurité II

Fehmi Jaafar Samuel Desbiens

Justin Bossard

Mattéo Gouhier

Paul Mathé

Samuel Plet

Léo Raclet

23 février 2026

Table des matières

1	Introduction	2
2	Gestion de Projet et Planification	3
2.1	Backlog et Division du travail	3
2.2	Diagramme de Gantt et Calendrier de réalisation	4
3	Frontend	5
3.1	Maquette conceptuelle du produit	5
3.2	Réalisation de l'interface	6
4	Backend	7
4.1	Structure	7
4.2	API	7
4.3	Prédiction	8
5	Architecture Backend : Le Moteur d'Analyse	9
5.1	Responsabilités du Module	9
5.2	Architecture Interne du Code	9
5.2.1	1. Le Point d'Entrée — <code>main.py</code>	9
5.2.2	2. Le Service de Communication IA — <code>ai_service.py</code>	9
5.3	Choix du Modèle	10
5.4	Flux de Traitement des Données	10
5.5	Format de Sortie	10
5.6	Sécurité & Performance	10
5.7	Perspectives d'Évolution	11

Partie 1

Introduction

Partie 2

Gestion de Projet et Planification

Pour assurer le bon déroulement du développement de **VerifAI** et respecter les échéances du plan de cours, nous avons opté pour une approche de gestion hybride. Nous utilisons l'outil **GitHub Projects**, qui nous permet de maintenir un backlog détaillé (méthodologie Agile) tout en générant automatiquement un diagramme de Gantt pour la planification macroscopique.

2.1 Backlog et Division du travail

Le projet a été découpé en tâches techniques précises. Pour répondre aux exigences de réalisation en équipe, chaque tâche de notre backlog est assignée à un responsable clair, garantissant une division équitable du travail.

Le tableau de bord utilise un système d'étiquettes (Labels) pour catégoriser les pôles d'expertise :

- **Développement Backend & Architecture (backend, archi, dev)** : Géré principalement par Mattéo GOUHIER (ex : mise en place de l'API FastAPI, codage du prototype MVP).
- **Interface & Expérience Utilisateur (frontend, design)** : Assuré par Paul MATHÉ (intégration de l'interface graphique) et Justin BOSSARD (création des maquettes).
- **Gestion & Recherche (gestion, recherche)** : Suivi administratif et bibliographique piloté par le reste de l'équipe pour assurer la conformité avec les jalons.
- **Données & Intelligence Artificielle (data, IA, science)** : Phase de sélection des datasets et protocoles préparatoires pour le modèle de détection.

Title	Labels	Assignees	Status	Linked pull requests	Sub-issues progress	Start	End	Milestone
1 Crée le nom de la startup et le logo #1	gestion	PumpeDie	Done			Jan 20, 2026	Jan 27, 2026	Jalon 1 : État de l'a
2 Compléter l'état de l'art sur la détection de Fake News (Recherche biblio) #2	recherche	Go-GoZ...	Done			Jan 20, 2026	Jan 27, 2026	Jalon 1 : État de l'a
3 Sélectionner les datasets (ex: LIAR, FakeNewsNet) #3	data	PumpeDie	Done			Jan 20, 2026	Jan 27, 2026	Jalon 1 : État de l'a
4 Choisir la stack technique (Python/Flask?, React?, PyTorch/tensorflow?) #4	archi	leoraclet	Done			Jan 20, 2026	Jan 27, 2026	Jalon 1 : État de l'a
5 Expérimentations possibles (supervisé vs outliars, persuasion IA) #21	test	paulm123...	Done			Jan 20, 2026	Jan 27, 2026	Jalon 1 : État de l'a
6 Définir le protocole d'expérimentation (métriques de succès) #5	science		Todo			Jan 20, 2026	Jan 27, 2026	Jalon 1 : État de l'a
7 Créer les maquettes conceptuelles (Figma/Adobe XD) #6	design	realistitaj	Todo			Jan 27, 2026	Feb 3, 2026	Jalon 2 : Prototype
8 Mettre en place l'API de base (squelette) #7	backend	leoraclet	Todo			Feb 3, 2026	Feb 24, 2026	Jalon 2 : Prototype
9 Intégration de l'interface graphique #8	frontend	paulm123...	Todo			Feb 3, 2026	Feb 24, 2026	Jalon 2 : Prototype
10 Concevoir le GANTT détaillé #9	gestion	PumpeDie	Done			Feb 3, 2026	Feb 24, 2026	Jalon 2 : Prototype
11 Codifier le prototype initial (MVP) #10	dev	leoraclet...	In Progress		0 / 3	Feb 3, 2026	Feb 24, 2026	Jalon 2 : Prototype
12 Connexion Front-Back #20			Todo			Feb 3, 2026	Feb 24, 2026	Jalon 2 : Prototype
13 Création des routes API (Login, Post News) #19			Todo			Feb 3, 2026	Feb 24, 2026	Jalon 2 : Prototype
14 Création de la base de données #18			Todo			Feb 24, 2026	Mar 31, 2026	Jalon 3 : IA & Mar
15 Implémentation du modèle de détection (Données temporelles) #11	IA		Todo			Feb 24, 2026	Mar 31, 2026	Jalon 3 : IA & Mar
16 Intégration du LLM pour l'explication automatisée des résultats #12	IA		Todo			Feb 24, 2026	Mar 31, 2026	Jalon 3 : IA & Mar
17 Rédiger l'étude de marché et identifier les concurrents #13	marketing		Todo			Feb 24, 2026	Mar 31, 2026	Jalon 3 : IA & Mar
18 Nettoyage et préparation finale des données d'entraînement #14	data		Todo			Feb 24, 2026	Mar 31, 2026	Jalon 3 : IA & Mar
19 Évaluation de la performance vs concurrents #15	test		Todo			Mar 31, 2026	Apr 28, 2026	Rendu Final : Procl
20 Tests de facilité d'utilisation #16	test		Todo			Mar 31, 2026	Apr 28, 2026	Rendu Final : Procl
21 Rédaction du rapport final complet #17	documentation		Todo			Mar 31, 2026	Apr 28, 2026	Rendu Final : Procl

FIGURE 2.1 : Vue Backlog - Division du travail et état d'avancement

Chaque tâche est associée à un Jalon cible (Milestone) et possède un statut de progression (*Todo, In Progress, Done*).

2.2 Diagramme de Gantt et Calendrier de réalisation

La vue chronologique de GitHub Projects nous permet de visualiser les dépendances et de paralléliser nos efforts. Le calendrier est structuré autour des quatre grandes phases du projet :

Phase 1 : Idéation et État de l'art (Jalon 1 - 20 Jan. au 27 Jan.) - *[Terminé]*

- Création de l'identité de la startup (Nom et Logo VerifAI).
- Recherche bibliographique et sélection du dataset (WELFake).
- Choix de la stack technique (Python, PyTorch, Vite).

Phase 2 : Prototypage et Maquettage (Jalon 2 - 27 Jan. au 24 Fév.) - *[En cours]* Cette phase concentre l'essentiel de nos efforts actuels de développement.

- **27 Jan - 03 Fév** : Création des maquettes conceptuelles de l'interface.
- **03 Fév - 24 Fév** : Développement en parallèle du Frontend (interface graphique) et du Backend (mise en place de l'API de base et création des routes de test).
- **Mi-Février** : Connexion Front-Back et validation du Prototype Minimum Viable (MVP).

Phase 3 : Intelligence Artificielle (Jalon 3 - 24 Fév. au 31 Mars) - *[À venir]* Une fois le prototype fonctionnel validé, l'effort basculera sur le moteur scientifique.

- Création de la base de données finalisée.
- Implémentation et entraînement du modèle de détection de Fake News.
- Intégration du LLM (Large Language Model) pour la génération automatique d'explications destinées aux utilisateurs.
- Rédaction de l'étude de marché marketing.

Phase 4 : Tests et Rendu Final (31 Mars au 28 Avril) - *[À venir]*

- Évaluation des performances du modèle face aux concurrents.
- Tests de facilité d'utilisation de la plateforme.
- Rédaction du rapport final complet et préparation de la soutenance.

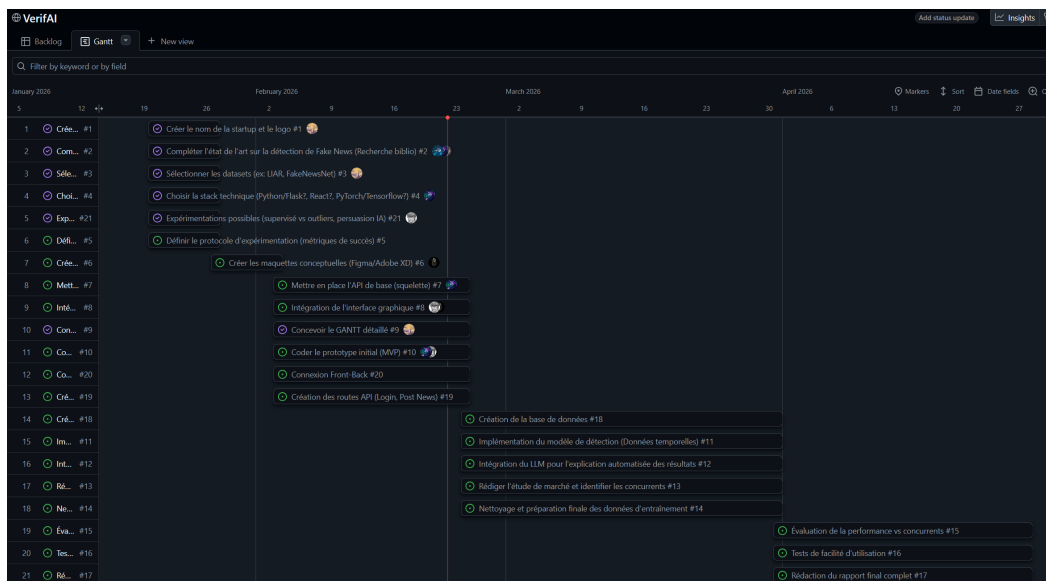


FIGURE 2.2 : Vue Gantt - Planification temporelle des Jalons

Partie 3

Frontend

3.1 Maquette conceptuelle du produit

Pour notre produit, nous avons cherché à avoir une interface simple et épuré, qui va à l'essentiel. Voilà la maquette conceptuelle de notre interface, en deux pages principales, accueil et résultats :

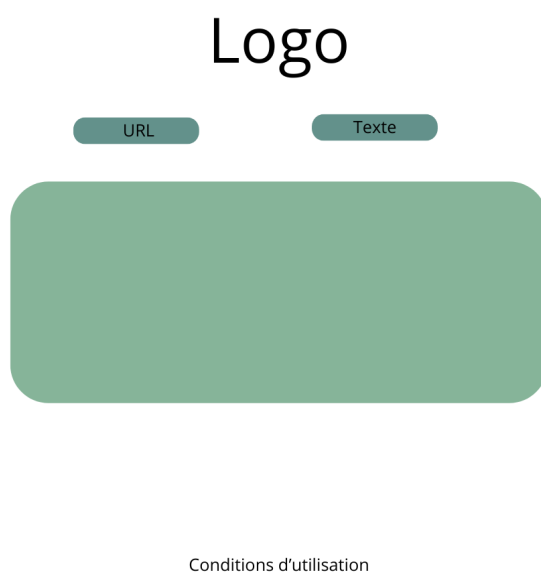


FIGURE 3.1 : Maquette conceptuelle de l'accueil de VerifAI

Logo

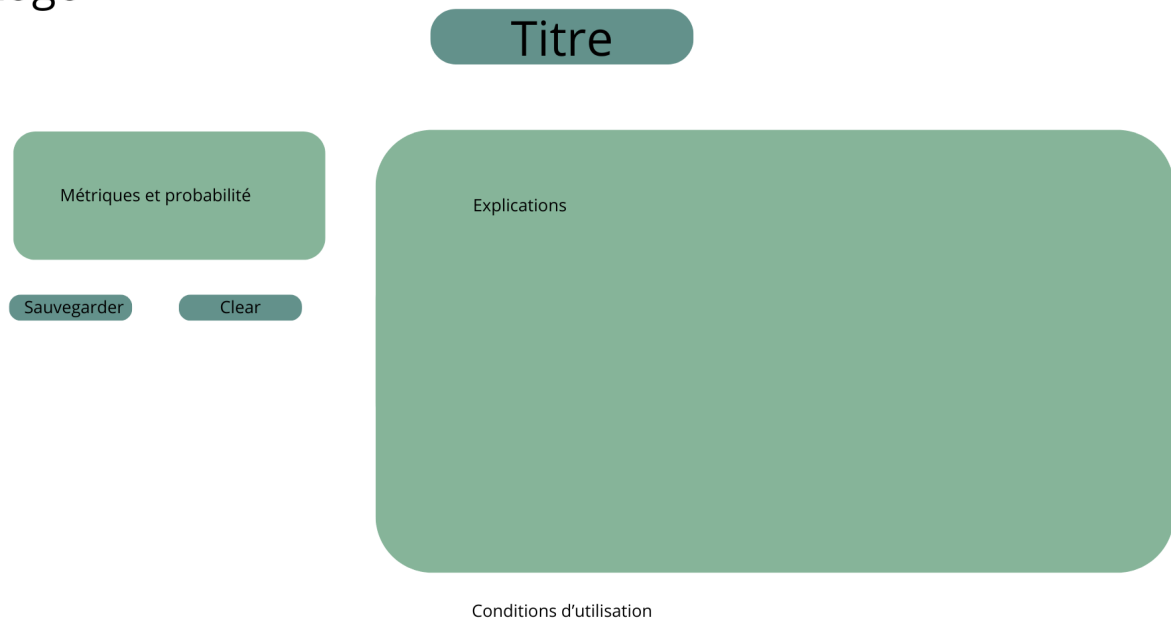


FIGURE 3.2 : Maquette conceptuelle des résultats

3.2 Réalisation de l'interface

Nous avons choisi d'utiliser [shadcn/ui](#) avec [Vite](#) pour réaliser notre produit : cela nous permet d'avoir simplement une application node, avec une interface minimaliste et simple à écrire.

Partie 4

Backend

Le backend met en œuvre une **API personnalisée** permettant notamment au frontend d'effectuer des actions telles que la vérification de texte ou le téléversement de fichiers.

4.1 Structure

L'image suivante présente l'architecture actuelle de notre solution. Elle montre comment s'articule le rôle de notre serveur au sein du système, ainsi que la manière dont s'organisent les interactions entre l'IA et l'utilisateur à travers celui-ci.

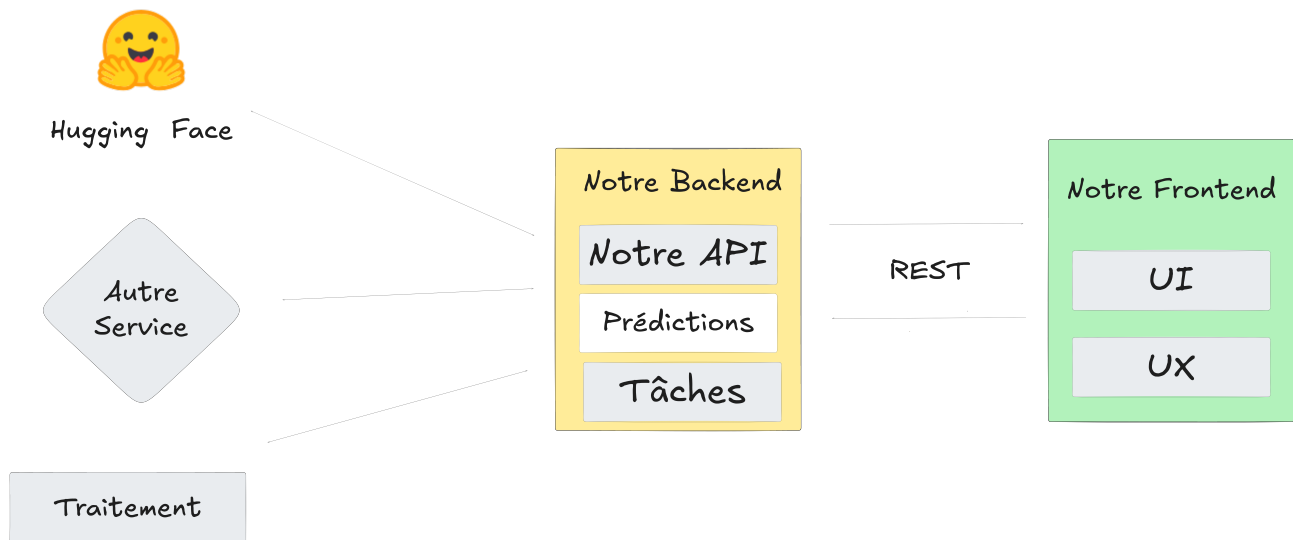


FIGURE 4.1 : Schema Technique du Projet

4.2 API

L'API est développée avec **FastAPI**, choisi pour :

- Son moteur asynchrone, idéal pour l'intégration d'IA et l'inférence.
- Sa compatibilité native avec **Pydantic**, qui facilite le typage des données.

Elle se structure autour de **deux catégories d'endpoints** :

- **/ai** : pour les prédictions et traitements liés à l'IA sur les données utilisateur.
- **/files** : pour la gestion de fichiers (images, PDF, etc.), notamment dans le cadre de la vérification de données provenant de sources variées.

4.3 Prédiction

Parmi les tâches assurées par le serveur figure l'inférence du modèle d'IA sur les données utilisateur. Ce processus repose sur un script Python qui effectue une requête vers l'API de Hugging Face, analyse la réponse obtenue, puis transmet les résultats au serveur, lequel les renvoie finalement à l'utilisateur.

Partie 5

Architecture Backend : Le Moteur d'Analyse

Le backend de cette application sert de **passerelle** entre les interfaces utilisateurs et les modèles de Deep Learning. Son rôle est de gérer la validation, la communication sécurisée et l'interprétation des données.

5.1 Responsabilités du Module

Le backend remplit trois fonctions principales :

1. **Sécurisation des secrets** : Gestion du Token API via variables d'environnement (`.env`).
 2. **Normalisation** : Transformation du texte brut en format compatible pour l'IA.
 3. **Traduction des résultats** : Transformer le retour via l'API en texte compréhensible par l'utilisateur.
-

5.2 Architecture Interne du Code

Le backend est segmenté en **deux composants** pour garantir la maintenabilité et éviter le code spaghetti.

5.2.1 1. Le Point d'Entrée — `main.py`

C'est lui qui contrôle l'application. Il reçoit les données entrantes, y applique les règles de validation (longueur minimale du texte), et construit la réponse finale suivant un schéma de réponse prédéfini.

5.2.2 2. Le Service de Communication IA — `ai_service.py`

Ce service est le point central de la connexion à l'IA :

- **Gestion des requêtes HTTP** : Utilisation de `requests` pour dialoguer avec le Cloud.
 - **Routage Dynamique** : Connexion au Router Hugging Face pour optimiser la disponibilité.
 - **Gestion de la résilience** : Paramétrage du `wait_for_model` pour éviter les plantages lors du chargement des modèles IA sur le serveur distant.
-

5.3 Choix du Modèle

Nous avons choisi un modèle réputé basé sur l'architecture **RoBERTa**, enrichi sur des articles comportant ou non des fake news. Ce choix n'est pas définitif, mais permet d'avoir une vision claire du fonctionnement de notre code.

Modèle utilisé : [hamzab/roberta-fake-news-classification](#)

5.4 Flux de Traitement des Données

Le backend transforme le texte brut en verdict compréhensible en **cinq étapes** :

Texte brut → Validation → Authentification → Analyse IA → Parsing → Post-traitement → JSON

1. Lecture du texte : Le système réceptionne le texte et valide son format (longueur minimale) pour filtrer les requêtes inutiles.
 2. Authentification : Il injecte de manière sécurisée la clé API pour autoriser la communication avec les serveurs de Hugging Face.
 3. Traitement par L'IA : Le texte est transmis au modèle RoBERTa, qui réalise une analyse sémantique profonde pour détecter les marqueurs de désinformation.
 4. Parsing : Le backend simplifie la réponse de l'IA (listes imbriquées) pour n'en extraire que les valeurs essentielles : le label et le score.
 5. Post-traitement : Il traduit ces statistiques en information humaine. Le score est converti en pourcentage et le label technique devient un verdict clair et lisible. Ce flux garantit que l'interface finale reçoit une donnée **fiable, lisible et prête à être affichée**.
-

5.5 Format de Sortie

Le backend ne renvoie pas de texte brut, mais un **objet JSON standardisé**. Cette structure permet de connecter n'importe quel Frontend sans modifier le code serveur.

```
{
  "is_fake": true,
  "confidence_score": 0.9997,
  "label_detected": "FAKE",
  "message": "Attention, ce contenu semble suspect.",
  "model_version": "hamzab/roberta-fake-news-classification"
}
```

5.6 Sécurité & Performance

Pour garantir la sécurité on stock les informations importante dans des variables d'environnement (dans .env) pour empêcher la fuite des clés API sur les dépôts de code publics. Et pour permettre d'améliorer la performance, nous avons construit l'application afin de pouvoir facilement changer le modèle (ici une instance de Roberta), à la seule condition de changer le format que l'on récupère sans le JSON.

5.7 Perspectives d'Évolution

Par la suite, nous pouvons penser changer le modèle pour prendre un modèle de type LLM, permettant de justifier le verdict. Cela permettrait d'afficher ces détails aux utilisateurs pour mieux comprendre les raisons derrière un résultat. On pourra le faire assez facilement, grâce à l'architecture expliquée au-dessus.